Practical Assignment Report

Practical Assignment 3: Talk Allocation

using a CHC Metaheuristic

Authors

Jesús Fernández López & Rafael Carlos Schoettel Vilchez

Subject

Metaheuristics

University of Córdoba

Date

April 17, 2026

Contents

1 Problem Analysis **2**
- 1.1 Context and Motivation . . . 2
- 1.2 Formal Problem Definition . . . 2
  - 1.2.1 Sets and Parameters . . . 2
  - 1.2.2 Entity Attributes . . . 2
  - 1.2.3 Decision Variable . . . 3
  - 1.2.4 Hard Constraints . . . 3
  - 1.2.5 Soft Constraints (Priorities) . . . 3

2 Design Decisions **4**
- 2.1 Chromosome Representation . . . 4
- 2.2 Preprocessing: Valid Researchers per Talk . . . 4

3 Fitness Function **5**
- 3.1 Minimisation Approach . . . 5
- 3.2 Modular Penalty Architecture . . . 5
  - 3.2.1 Hard Constraint Penalties . . . 5
  - 3.2.2 Soft Constraint Penalties . . . 6
- 3.3 Configuration Dictionary . . . 6

4 Metaheuristic Logic **6**
- 4.1 Overview of CHC . . . 6
- 4.2 HUX Crossover . . . 7
- 4.3 Incest Prevention and the Hamming Distance Threshold . . . 7
- 4.4 Cataclysmic Restart . . . 8
- 4.5 Algorithm Flow . . . 8
- 4.6 Parameter Configuration . . . 9

5 Conclusions **9**

1 Problem Analysis

1.1 Context and Motivation

Every year, to celebrate the *International Day of Women and Girls in Science* (February 11th), the University of Córdoba organises a programme of scientific outreach talks at local schools. Schools from the city and the surrounding province submit requests specifying the educational level and preferred topic of the desired talk. At the same time, researchers volunteer to deliver talks within their area of expertise, subject to constraints on travel and educational level.

Once both sets of requests are received, a coordinator must match talks to researchers. The matching must satisfy hard feasibility requirements while optimising a set of priority-based soft objectives. The number of researchers R and the number of requested talks T vary each year, producing two qualitatively different regimes that the algorithm must handle transparently.

1.2 Formal Problem Definition

1.2.1 Sets and Parameters

Symbol	Definition
T	Total number of talks requested by schools
E	Total number of schools
R	Total number of researchers
$\mathcal{T} = \{t_0, \ldots, t_{T-1}\}$	Set of requested talks
$\mathcal{S} = \{s_1, \ldots, s_E\}$	Set of schools
$\mathcal{R} = \{r_1, \ldots, r_R\}$	Set of researchers

1.2.2 Entity Attributes

Each **school** $s \in \mathcal{S}$ is characterised by:

- $\text{loc}(s) \in \{\text{city}, \text{province}\}$ — geographical location.
- $\text{disadv}(s) \in \{0, 1\}$ — whether s is in a disadvantaged area.
- $\text{type}(s) \in \{\text{public}, \text{concerted}, \text{private}\}$ — institution type.
- $\text{new}(s) \in \{0, 1\}$ — whether s participates for the first time.

Each **talk request** $t \in \mathcal{T}$ carries:

- $\text{school}(t) \in \mathcal{S}$ — the requesting school.
- $\text{topic}(t) \in \mathcal{K} \cup \{\text{any}\}$ — required topic ($\mathcal{K}$ is the set of known disciplines; "any" means no restriction).
- $\text{level}(t) \in \mathcal{L}$ — required educational level (preschool, primary, secondary, high school, or vocational training).

Each **researcher** $r \in \mathcal{R}$ is described by:

- $\text{topic}(r) \in \mathcal{K}$ — area of expertise.
- $\text{level}(r) \in \mathcal{L}$ — preferred educational level.
- $\text{travel}(r) \in \{0, 1\}$ — whether r can travel outside the city.
- $\text{newR}(r) \in \{0, 1\}$ — first year participating.
- $\text{prevProv}(r) \in \{0, 1\}$ — gave a talk in the province last year.
- $\text{prevSchool}(r) \in \mathcal{S} \cup \{\varnothing\}$ — school visited last year.
- $\text{maxTalks}(r) \in \{1, 2\}$ — maximum talks r is willing to give.

1.2.3 Decision Variable

The problem is equivalent to a **Generalised Assignment Problem** (GAP). The decision variable is an assignment function:

$$x : \mathcal{T} \to \mathcal{R} \cup \{-1\} \tag{1}$$

where $x(t) = r$ means talk t is assigned to researcher r, and $x(t) = -1$ means talk t is left unassigned.

1.2.4 Hard Constraints

HC1. Topic compatibility: $\text{topic}(t) = \text{any}$ *or* $\text{topic}(r) = \text{topic}(t)$, for all assigned (t, r).

HC2. Level compatibility: $\text{level}(r) = \text{level}(t)$, for all assigned (t, r).

HC3. Coverage when $R \geq T$: Every school s must have at least one assigned talk: $\exists\, t\colon \text{school}(t) = s,\ x(t) \neq -1$.

HC4. Location feasibility: If $\text{loc}(\text{school}(t)) = \text{province}$ then $\text{travel}(x(t)) = 1$.

HC5. Capacity: $|\{t : x(t) = r\}| \leq \text{maxTalks}(r)$ for all r.

1.2.5 Soft Constraints (Priorities)

When $T > R$ (insufficient researchers), schools are prioritised in the following order:

1. School participates for the first time.
2. School is located in the province.
3. School is in a disadvantaged area.
4. School is a public institution.
5. School is concerted.
6. School is private (lowest priority).

When $R > T$ (surplus researchers), researchers are prioritised:

1. Researcher participates for the first time.
2. Researcher can travel to the province.

Additional historical soft constraints:

- Researcher who was in the province last year should preferably go to the city.
- Researcher should preferably not revisit the same school as last year.

2 Design Decisions

2.1 Chromosome Representation

We encode a solution as a one-dimensional integer array of length T:

$$\mathbf{c} = [c_0,\, c_1,\, \ldots,\, c_{T-1}], \quad c_i \in \{-1,\, 0,\, \ldots,\, R-1\} \tag{2}$$

Position i of the chromosome corresponds to talk t_i. The value c_i is the *index* of the researcher assigned to that talk (or -1 for unassigned). This **direct encoding** has several desirable properties:

- Compact representation: T genes, each with at most $R+1$ possible values.
- Simple crossover: any valid gene-level operator is applicable.
- Transparent decoding: solution quality is immediately readable from the array.

2.2 Preprocessing: Valid Researchers per Talk

Before launching the metaheuristic, we pre-compute for each talk t the set $V(t)$ of researchers that can feasibly cover it (stored as a Python dictionary called `valid_researchers_per_talk`):

$$\begin{aligned} V(t) = \{\, r \in \mathcal{R} \mid\ & (\text{topic}(t) = \text{any} \lor \text{topic}(r) = \text{topic}(t)) \\ & \land\ \text{level}(r) = \text{level}(t) \\ & \land\ (\text{loc}(\text{school}(t)) = \text{city} \lor \text{travel}(r) = 1)\,\} \end{aligned}$$

This **mandatory preprocessing step**:

- Enforces hard constraints HC1–HC4 *before* the search begins, drastically reducing the effective search space.
- Speeds up initialisation: random chromosomes are built by sampling from $V(t)$, producing feasible or near-feasible individuals from generation 0.
- Makes the fitness function lighter: constraint HC1–HC4 violations can be detected in $O(1)$ using the precomputed set.

Talks for which $V(t) = \varnothing$ are inherently infeasible; such genes are set to -1 and are never modified by crossover or mutation.

```python
def build_valid_researchers_per_talk(talks, researchers, schools):
    valid = {}
    for talk in talks:
        school = schools[talk.school_id]
        candidates = []
        for r in researchers.values():
            # Hard constraint 1: topic match
            if talk.topic != "any" and r.topic != talk.topic:
                continue
            # Hard constraint 2: level match
            if r.level != talk.level:
                continue
            # Hard constraint 3: travel feasibility
            if school.location == "province" and not r.can_travel:
                continue
            candidates.append(r.researcher_id)
        valid[talk.talk_id] = candidates
    return valid
```

Listing 1: Preprocessing: building the valid researchers dictionary

3 Fitness Function

3.1 Minimisation Approach

The fitness function measures the *total penalty* of a chromosome. Following a minimisation paradigm, a perfect solution has $f(\mathbf{c}) = 0$, and each unsatisfied constraint contributes a positive penalty proportional to its weight:

$$f(\mathbf{c}) = \underbrace{\sum_k w_k^{\text{hard}} \cdot \delta_k^{\text{hard}}(\mathbf{c})}_{\text{hard penalties}} + \underbrace{\sum_k w_k^{\text{soft}} \cdot \delta_k^{\text{soft}}(\mathbf{c})}_{\text{soft penalties}} \tag{3}$$

where $\delta_k(\mathbf{c}) \in \{0, 1, \mathbb{Z}^+\}$ counts violations of constraint k.

3.2 Modular Penalty Architecture

Each constraint class has its own function, collecting penalties independently and summing them. This modularity simplifies testing, enables weight tuning, and allows constraints to be enabled or disabled individually.

3.2.1 Hard Constraint Penalties

Table 1: Hard constraint penalties and default weights.

Violation	Description	Default Weight
School without talk ($R \geq T$)	A school receives zero talks when there are enough researchers	1,000,000
Location mismatch	Researcher without travel clearance assigned to a province school	1,000,000
Overallocation	Researcher assigned more talks than their `maxTalks` limit	1,000,000

Setting hard penalties to 10^6 ensures that any feasible solution — even one that satisfies no soft constraint — outranks any infeasible solution.

3.2.2 Soft Constraint Penalties

Table 2: Soft constraint penalties, their context, and default weights.

Violation	Description	Default Weight
First-year school unserved	School in its first year receives no talk	500
Province school unserved	Province school receives no talk ($T > R$)	300
Disadvantaged area unserved	Disadvantaged school receives no talk	400
Public school unserved	Public institution receives no talk	200
Concerted school unserved	Concerted school receives no talk	100
First-year researcher unused	New researcher not selected ($R > T$)	200
Traveller unused in province	Travelling researcher not used for province ($R > T$)	150
Same school as last year	Researcher revisits their school from last year	300
Same province as last year	Province researcher goes to province again	150

3.3 Configuration Dictionary

All weights are collected in a single Python dict (`DEFAULT_CONFIG`). This makes the fitness function *fully configurable*: a user can pass a custom dict to `compute_fitness()` to reflect different institutional priorities without modifying any source code.

```python
DEFAULT_CONFIG = {
    # Hard constraints
    "w_school_no_talk":     1_000_000,
    "w_location_mismatch":  1_000_000,
    "w_overallocation":     1_000_000,
    # Soft: unserved schools
    "w_first_year_school":  500,
    "w_province_school":    300,
    "w_disadvantaged":      400,
    "w_public_institution": 200,
    "w_concerted":          100,
    # Soft: unused researchers (R > T regime)
    "w_first_year_researcher": 200,
    "w_travel_researcher":     150,
    # Soft: historical
    "w_same_school_as_last_year":   300,
    "w_same_province_as_last_year": 150,
}
```

Listing 2: Configurable weight dictionary

4 Metaheuristic Logic

4.1 Overview of CHC

CHC (Cross-generational elitist selection, Heterogeneous recombination, Cataclysmic mutation) is an evolutionary algorithm proposed by Eshelman (1991) that combines

aggressive elitism with operators designed to maintain population diversity. Its three defining mechanisms are:

1. **Cross-generational elitism (C):** All individuals — old and new — compete for survival; only the best N_p are kept.
2. **HUX crossover (H):** A specialised operator that maximises diversity in offspring while preserving strong genes from both parents.
3. **Cataclysmic restart (C):** When the population has converged completely, a controlled restart diversifies around the best individual.

Unlike standard GAs, CHC uses *no mutation during the main loop*. Instead, incest prevention prevents premature convergence, and the restart provides macro-level diversity when the incest threshold reaches zero.

4.2 HUX Crossover

The Half-Uniform Crossover operator is the core recombination method in CHC. Given two parents $\mathbf{p}_1$ and $\mathbf{p}_2$, HUX:

1. Identifies the set D of positions where p_1 and p_2 differ: $D = \{i : c_i^{(1)} \neq c_i^{(2)}\}$.
2. Randomly selects exactly $\lfloor |D|/2 \rfloor$ positions from D to swap.
3. Produces two offspring $\mathbf{c}_1$, $\mathbf{c}_2$ by exchanging the selected genes between parents.

$$|D(\mathbf{c}_1, \mathbf{c}_2)| = |D(\mathbf{c}_1, \mathbf{p}_1)| = |D(\mathbf{c}_2, \mathbf{p}_2)| = \left\lfloor \frac{|D(\mathbf{p}_1, \mathbf{p}_2)|}{2} \right\rfloor \tag{4}$$

This property guarantees that each offspring is exactly halfway between the two parents in Hamming space, which is the furthest apart they can be while each carrying a mix of both parents. HUX maximises exploration among the offspring of a given pair.

```python
def hux_crossover(parent1, parent2):
    diff_positions = [i for i, (a, b) in enumerate(zip(parent1, parent2)
    ) if a != b]
    if len(diff_positions) < 2:
        return copy.copy(parent1), copy.copy(parent2)
    half = len(diff_positions) // 2
    swap_positions = set(random.sample(diff_positions, half))
    child1, child2 = copy.copy(parent1), copy.copy(parent2)
    for pos in swap_positions:
        child1[pos], child2[pos] = child2[pos], child1[pos]
    return child1, child2
```

Listing 3: HUX crossover implementation

4.3 Incest Prevention and the Hamming Distance Threshold

A key feature of CHC is the **incest prevention** mechanism. Two individuals are only allowed to mate if their Hamming distance exceeds a dynamic threshold d:

$$d_H(\mathbf{p}_1, \mathbf{p}_2) = \sum_{i=0}^{T-1} \mathbf{1}[c_i^{(1)} \neq c_i^{(2)}] > d \tag{5}$$

The threshold d is initialised to $\lfloor T/4 \rfloor$ and decreases by 1 each generation in which no offspring improves over both parents. This progressive relaxation allows the algorithm to increase mating pressure as the population converges, eventually reaching all pairs.

Elitist replacement: Children only enter the population if they strictly improve over at least one of their parents. This cross-generational competition ensures monotonic improvement: the best fitness never decreases.

4.4 Cataclysmic Restart

When d reaches 0 and no crossover produces improvements, the population has converged. Rather than terminating, CHC performs a **cataclysmic restart**:

1. The current best individual (the *survivor*) is copied unchanged.
2. The remaining $N_p - 1$ individuals are created by randomly mutating exactly $\lfloor \mu \cdot T \rfloor$ genes of the survivor ($\mu = 35\%$ by default).
3. Each mutated gene is replaced by a valid researcher drawn uniformly from $V(t)$ for that position.
4. The Hamming threshold is reset to $\lfloor T/4 \rfloor$.

This restart is *controlled*: the best solution is never lost, but the population is aggressively diversified around it, enabling further exploration in new regions.

```python
def cataclysmic_mutation(best, pop_size, mutation_rate, talks, valid_map, r_id_list):
    new_population = [copy.copy(best)]
    n_mutate = max(1, int(mutation_rate * len(talks)))
    for _ in range(pop_size - 1):
        mutant = copy.copy(best)
        positions = random.sample(range(len(talks)), n_mutate)
        for pos in positions:
            mutant[pos] = repair_gene(pos, valid_map, r_id_list)
        new_population.append(mutant)
    return new_population
```

Listing 4: Cataclysmic restart

4.5 Algorithm Flow

Algorithm 5 summarises the complete CHC loop.

```
threshold = T // 4
population = initialise_population(pop_size)
while generation < max_generations:
    shuffle(population)
    pairs = [(pop[i], pop[i+1]) for i in range(0, pop_size-1, 2)]
    new_individuals = []
    for p1, p2 in pairs:
        if hamming(p1, p2) > threshold:
            c1, c2 = hux_crossover(p1, p2)
            if fitness(c1) < min(fitness(p1), fitness(p2)):
                new_individuals.append(c1)
            if fitness(c2) < min(fitness(p1), fitness(p2)):
                new_individuals.append(c2)
    if new_individuals:
        merge and keep top pop_size individuals   # cross-generational elitism
    else:
        threshold -= 1                            # tighten incest threshold
    if threshold <= 0:
        population = cataclysmic_restart(best)   # reset around best
        threshold = T // 4
    generation += 1
```

Listing 5: CHC main loop (simplified)

4.6 Parameter Configuration

Table 3: CHC hyperparameters used in this implementation.

Parameter	Symbol	Default Value
Population size	N_p	50
Maximum generations	G	200
Initial threshold	d_0	$\lfloor T/4 \rfloor$
Restart mutation rate	μ	35%

The choice of $\mu = 35\%$ follows Eshelman's recommendation: too small a mutation produces restarts that are virtually identical to the best, while too large a value effectively discards the best solution's genetic information.

5 Conclusions

This practice addressed the **Talk Allocation Problem** as a Generalised Assignment Problem with one hard categorical constraint per dimension (topic, level, location) and a rich set of lexicographic soft priorities. The CHC metaheuristic was selected because it naturally balances intense elitism with controlled diversity preservation, without relying on hand-tuned mutation probabilities during the main evolutionary loop.

The modular penalty architecture allows practitioners at the University of Córdoba to adjust the relative importance of each priority rule — for example, giving more weight to disadvantaged areas in a particular year — without touching the algorithm's source code.

The mandatory preprocessing phase (`valid_researchers_per_talk`) reduces feasibility enforcement to an $O(1)$ lookup during fitness evaluation, making the algorithm scalable to larger real-world instances with hundreds of talks and researchers.

Future work could introduce a *repair operator* that, whenever a child violates a hard constraint on a gene, replaces that gene with a draw from $V(t)$ before evaluating fitness. This would further accelerate convergence by keeping the population closer to the feasible region at all times.